The Chatman Equation:

A Receipt Calculus for Reality-Addressed Agentic AI

Sean Chatman
Independent Researcher

Genesis — Foundations, Paper 00

Chapter 0: Abstract

This thesis program paper presents a formal receipt calculus designed to establish verifiable accountability in autonomous agentic AI systems. We address the fundamental gap between the scale of model-driven execution traces and the cognitive limits of human operators ($\kappa \approx 4$).

Algebraic Foundation. We factor the raw, undecidable observation space $\mathcal{O}$ by introducing the extended observation space $\mathcal{O}_\perp = \mathcal{O} \cup \{\perp\}$ and defining a computable retraction map $\alpha : \mathcal{O}_\perp \to \mathcal{O}^* \cup \{\perp\}$. This retraction maps raw, unstructured inputs onto a decidable, normalized syntactic subspace $\mathcal{O}^*$ while mapping validation failures to the first-class refusal value $\perp$. Refusal composition is formalized via the denial monoid, a commutative idempotent join-semilattice. Typestate and execution order are modeled category-theoretically using a path category over a linear quiver, where illegal transitions map to empty hom-sets.

Geometric & Process Modeling. We represent lifecycle trajectories and process execution. The lifecycle process is mapped to a safe Petri net token game, enabling the evaluation of conformance fitness $\varphi \in [0, 1]$ during replay. We model the manufacturing morphism $\mu : \mathcal{O}^* \cup \{\perp\} \to \mathcal{A} \cup \{\perp\}$ as a deterministic, cost-bounded map that translates admitted payloads into actuated artifacts while propagating refusal.

Cryptographic Commitment. Execution traces are projected onto a compact receipt space $\mathcal{R}$. Each receipt contains a collision-resistant hash digest $h_+ = \mathcal{H}(h_- \parallel \mathsf{fr})$, where the causal frame fr commits recursively to the timestamp, instruction, and payload bytes.

Limits & Guarantees. Under the assumption of the collision resistance of $\mathcal{H}$ and the completeness of the host compiler's type checking, we prove the Conservation of Consequence. This theorem guarantees that every actuation event is mapped injectively to a unique position in the receipt chain, and the corresponding actuated artifact is caused by at least one admitted observation. Importantly, our proofs are bounded by these cryptographic and type-system assumptions; we do not claim absolute semantic correctness of model outputs, but rather verify that actuation conforms strictly to the specified syntactic and lifecycle constraints. The resulting framework provides a rigorous, constant-time audit boundary for complex autonomous systems.

Contents

Chapter 1

Chapter 1: The Problem: Trust Without Comprehension

1.1 The Question

How can a human agent verify the behavior of an autonomous agentic AI system when the system's interior state space exceeds human cognitive capacity?

1.2 Why the Question is Forced

This question is forced because human cognitive capacity is a fixed, small biological constant. A system whose interior execution trace cannot be held in human working memory cannot be trusted via direct runtime comprehension. Therefore, we must find an alternative verification mechanism that does not require full semantic comprehension.

1.3 Definitions and Constructions

Definition 1.1 (Cognitive Capacity Limit)**.** The cognitive capacity limit $\kappa \in \mathbb{N}$ represents the maximum number of independent information chunks a human operator can hold in working memory. Canonically, we assume $\kappa \approx 4$.

Construction 1.1 (Comprehension-Verification Gap)**.** Let $\sigma \in \Sigma$ be an execution trace of size n. The cognitive cost to comprehend the interior of the system is modeled as $\mathrm{Comp}(\sigma) = \Theta(\dim \Sigma)$. The cost to verify the boundary projection is modeled as $\mathrm{Ver}_\partial(\sigma) = O(\kappa)$. The ratio of comprehension cost to verification cost is the comprehension-verification gap:

$$\frac{\mathrm{Comp}(\sigma)}{\mathrm{Ver}_\partial(\sigma)} = \Omega\left(\frac{\dim \Sigma}{\kappa}\right)$$

which diverges to infinity as the complexity of the trace grows.

Theorem 1.1 (Divergence of Comprehension)**.** *As trace dimension grows, verification cost remains bounded while trace dimension grows, assuming the receipt projection remains faithful and the hash assumption holds.*

Proof. By Construction 1.1, $\mathrm{Ver}_{\partial}(\sigma)$ is bounded by $O(\kappa)$ which is constant. Since $\dim \Sigma$ is unbounded as trace length increases, the ratio $\dim \Sigma / \kappa$ diverges to infinity. Hence, direct comprehension of the trace is impossible for large systems, forcing reliance on a constant-sized boundary projection. □

1.4 Operational Correspondence

In the `praxis` implementation, this gap is operationalized by providing signature and hash verification via the `verify` verb, which checks a fixed-size receipt tuple rather than executing full RDF graph re-evaluations.

1.5 Boundaries

We do not verify the semantic intent of the AI agent; we restrict verification to the boundary receipts.

Chapter Summary.

- **What was admitted:** The cognitive capacity limit κ and the comprehension-verification gap.
- **What was refused:** Unbounded runtime human comprehension of execution traces.
- **What this enables next:** The introduction of the Chatman Equation in Chapter 2 to factor the execution path.

Chapter 2

Chapter 2: The Equation: Five Objects and One Route

2.1 The Question

What algebraic objects and mappings are necessary to factor an agentic system's execution path into verifiable components?

2.2 Why the Question is Forced

This question is forced because we cannot execute raw observations directly due to Rice's Theorem. We must factor the execution route so that validation is performed on a decidable sub-language before manufacture.

2.3 Definitions and Constructions

Definition 2.1 (Observation Spaces and Refusal Constant)**.** Let $\mathcal{O}$ be the raw observation space containing arbitrary finite byte strings. Let $\perp$ be the distinguished refusal constant representing failure, halt, or invalid input, where $\perp \notin \mathcal{O}$. The extended observation space $\mathcal{O}_\perp$ is the union:

$$\mathcal{O}_\perp = \mathcal{O} \cup \mathcal{H} \cup \{\perp\}$$

where $\mathcal{H}$ is the refusal space of halted configurations defined in Chapter 4. Let $\mathcal{O}^* \subseteq \mathcal{O}$ be the admitted syntactic subspace consisting of raw observations that pass all obligations.

Definition 2.2 (Refined Chatman Equation)**.** This paper studies one architecture in which scalable action standing is obtained by decidable admission, bounded manufacture, and replayable receipts. The governing equation is:

$$\boxed{\mathcal{A} \;=\; \mu(\mathcal{O}^*)} \tag{2.1}$$

where $\mathcal{A}$ is the action space representing actuated system changes. The admission map is a function $\alpha : \mathcal{O}_\perp \to \mathcal{O}^* \cup \{\perp\}$ whose image is $\mathrm{im}(\alpha) = \mathcal{O}^* \cup \{\perp\}$. The manufacturing morphism is a map $\mu : \mathcal{O}^* \to \mathcal{A}$, which is extended to the domain $\mathcal{O}^* \cup \{\perp\}$ by propagating the refusal constant such that $\mu(\perp) = \perp$.

Construction 2.1 (The Five Objects)**.** The Chatman equation relates five spaces and maps:

(i) $\mathcal{O}$, the raw observation space containing arbitrary finite byte strings.

(ii) $\mathcal{O}^* \subseteq \mathcal{O}$, the admitted syntactic subspace.

(iii) $\mu : \mathcal{O}^* \to \mathcal{A}$, the manufacturing morphism, extended as $\mu : \mathcal{O}^* \cup \{\bot\} \to \mathcal{A} \cup \{\bot\}$ with $\mu(\bot) = \bot$.

(iv) $\mathcal{A}$, the artifact/action space representing actuated system changes.

(v) $\mathcal{R}$, the receipt space containing bounded execution projections.

Theorem 2.1 (Morphism Domain Consistency)**.** *The manufacturing morphism μ is well-defined on all elements of the admitted space $\mathcal{O}^*$ and propagates the refusal constant $\bot$ without evaluating raw payloads.*

Proof. By Definition 2.2, μ maps $\mathcal{O}^*$ to $\mathcal{A}$. For $x \in \mathcal{O}^*$, $\mu(x)$ is a well-defined element of $\mathcal{A}$. For $x = \bot$, the map propagates to $\bot$. Since $\mathcal{O} \setminus \mathcal{O}^*$ is excluded from the domain of μ (being mapped to $\bot$ by the admission map α), no unadmitted raw observation bypasses the admission filter. □

2.4 Operational Correspondence

The five objects are mapped to the Rust type family `LawObject<Payload, S: Stage, Law>` in `crates/praxis-core/src/law.rs`, which enforces transitions between stages.

2.5 Boundaries

The manufacturing morphism μ cannot be evaluated on elements of $\mathcal{O} \setminus \mathcal{O}^*$.

Chapter Summary.

- **What was admitted:** The signature of μ on the admitted domain $\mathcal{O}^*$.
- **What was refused:** Evaluating the manufacturing morphism on raw, unvalidated observations.
- **What this enables next:** Analyzing the computability boundary of semantic validation in Chapter 3.

Chapter 3

Chapter 3: The Computability Boundary: Why Admission Exists

3.1 The Question

Why is it mathematically impossible to design an admission gate that decides the semantic correctness of an observation?

3.2 Why the Question is Forced

This question is forced by Rice's Theorem on recursive enumerability. Any attempt to decide a non-trivial semantic property of program behavior within a total program will fail, forcing us to restrict admission to syntactic retractions.

3.3 Definitions and Constructions

Axiom 3.1 (Observation Space)**.** There is a set $\mathcal{O}$, the observation space, whose elements are arbitrary finite records. $\mathcal{O}$ carries no decidable semantics; the predicate "does this observation mean what it purports to mean" is not assumed computable.

Theorem 3.1 (Rice, specialized to observations)**.** *Let $\mathcal{U}$ be a universal model of computation and let observations in $\mathcal{O}$ range over finite encodings that may denote arbitrary $\mathcal{U}$-programs. Let P be any non-trivial semantic property of those denoted functions. Then the set $\{o \in \mathcal{O} : P(o)\}$ is undecidable.*

Proof. This is Rice's theorem [2] instantiated at $\mathcal{O}$. Suppose a decider D existed for $\{o : P(o)\}$. Fix observations o_+ with P and o_- without P. Given any machine M and input x, build the observation $o_{M,x}$ denoting the program: "run $M(x)$; if it halts, behave as $e(o_+)$; else diverge." Then $o_{M,x}$ denotes a function with property P iff $M(x)$ halts. Thus, $D(o_{M,x})$ decides the halting problem, which is a contradiction. Hence, no such decider D exists. □

Corollary 3.2 (Admission cannot be semantic decision)**.** *There is no algorithm that admits an observation $o \in \mathcal{O}$ by deciding a non-trivial semantic property of what o denotes. Any total, terminating admission procedure must therefore decide a syntactic, decidable surrogate.*

Proof. Immediate from Theorem 3.1. A total admission map that decided a non-trivial semantic property would decide an undecidable set, which is a contradiction. □

3.4 Operational Correspondence

The `praxis` parser handles raw payloads as untyped JSON strings using `serde_json::Value` in `src/verbs/law.rs`, which does not execute code during initial parsing.

3.5 Boundaries

We do not use terms like "understands intent" or "proves meaning"; validation is strictly confined to syntactic conformance.

Chapter Summary.

- **What was admitted:** The syntactic boundary as a decidable surrogate for semantic validation.
- **What was refused:** Semantic verification of program behavior at ingress.
- **What this enables next:** The definition of the admission algebra and refusal monoid in Chapter 4.

Chapter 4

Chapter 4: Refusal as Data: The Admission Algebra

4.1 The Question

How can validation failures be composed and computed as first-class data without raising exceptions, and how is this executed at register speed?

4.2 Why the Question is Forced

This question is forced because we must maintain total execution flow in high-frequency environments. Exceptions bypass formal state tracking, so we must represent refusal as a value in a commutative idempotent monoid.

4.3 Definitions and Constructions

Definition 4.1 (Extended Observation Space)**.** Let $\mathcal{O}$ be the raw observation space. Let $D_{\text{refuse}} = \{0,1\}^n$ be the space of denial words representing obligation failures. We formalize the refusal space as the set of halted configurations:

$$\mathcal{H} = \mathcal{O} \times D_{\text{refuse}} \times \mathbb{N}$$

where an element $(o, d, t) \in \mathcal{H}$ carries the original raw observation o, the non-zero denial word d, and a timestamp $t \in \mathbb{N}$ representing when the failure occurred. The extended observation space $\mathcal{O}_\perp$ is defined as:

$$\mathcal{O}_\perp = \mathcal{O} \cup \mathcal{H} \cup \{\perp\}$$

where $\perp$ is the distinguished refusal constant representing terminal failure or lack of any input.

Definition 4.2 (Admission Map)**.** Let $\mathcal{O}^* \subseteq \mathcal{O}$ be the admitted space. Let $g_1, \ldots, g_m : \mathcal{O} \to \{0,1\}$ be total computable obligations. The admission map $\alpha : \mathcal{O}_\perp \to \mathcal{O}^* \cup \{\perp\}$ is:

$$\alpha(x) = \begin{cases} \rho(o) \in \mathcal{O}^* & \text{if } x = o \in \mathcal{O} \text{ and } \bigwedge_{i=1}^{m} g_i(o) = 1, \\ \perp & \text{otherwise,} \end{cases}$$

where $\rho : \mathcal{O} \rightharpoonup \mathcal{O}^*$ is a computable normalization fixing $\mathcal{O}^*$ pointwise. To ensure α is total, we assume the obligation compatibility condition:

$$\{o \in \mathcal{O} : \bigwedge_{i=1}^{m} g_i(o) = 1\} \subseteq \mathrm{dom}(\rho)$$

reflecting that any observation passing all obligations is within the domain of the normalization map.

Theorem 4.1 (Idempotence of Admission)**.** *The admission map is a retraction: $\alpha \circ \alpha = \alpha$ on all of $\mathcal{O}_\perp$.*

Proof. For $x = \perp$, $\alpha(\alpha(\perp)) = \alpha(\perp) = \perp$. For $x \in \mathcal{H}$, $\alpha(x) = \perp$ by definition, so $\alpha(\alpha(x)) = \alpha(\perp) = \perp$. For $x = o \in \mathcal{O}$ where obligations fail, $\alpha(o) = \perp$, so $\alpha(\alpha(o)) = \alpha(\perp) = \perp$. For $x = o \in \mathcal{O}$ where obligations pass, $y = \alpha(o) = \rho(o) \in \mathcal{O}^* \subseteq \mathcal{O}$. Since ρ fixes $\mathcal{O}^*$ pointwise and obligations pass on $\mathcal{O}^*$, $\alpha(y) = y$. Hence, $\alpha(\alpha(x)) = \alpha(x)$ for all inputs in $\mathcal{O}_\perp$. □

Construction 4.1 (Denial Monoid)**.** Let $D = (\{0,1\}^n, \vee, \mathbf{0})$ be the commutative idempotent monoid of denial words of width n, where $\mathbf{0}$ represents a clean admission and $\vee$ is bitwise OR. For a subset of active obligations $G \subseteq \{1, \ldots, n\}$, and an observation $o \in \mathcal{O}$, let $d_i(o) \in \{0,1\}$ be the binary indicator of the failure of obligation i (where $d_i(o) = 1$ if obligation i is violated, and 0 otherwise). The denial word projection $d_G(o) \in D$ is defined as the bit-vector whose components corresponding to indices in G are $d_i(o)$ and whose other components are 0, so that:

$$d_G(o) = \bigvee_{i \in G} d_i(o)\mathbf{e}_i$$

where $\mathbf{e}_i$ is the i-th standard basis vector of $\{0,1\}^n$.

Proposition 4.2 (Monotonicity of Denial)**.** *Adding obligations to the battery can only enlarge the denial word under the lattice join, i.e., $\{o : d_{G'}(o) = \mathbf{0}\} \subseteq \{o : d_G(o) = \mathbf{0}\}$ for $G \subseteq G'$.*

Proof. Let $d_G(o) = \bigvee_{i \in G} d_i(o)$. For $G' = G \cup \Delta$, $d_{G'}(o) = d_G(o) \vee \bigvee_{i \in \Delta} d_i(o)$. Under the Boolean lattice, $d_G(o) \preceq d_{G'}(o)$. If $d_{G'}(o) = \mathbf{0}$, then $d_G(o) = \mathbf{0}$ must hold. □

Definition 4.3 (Refusal Taxonomy)**.** A refusal taxonomy is a total map $\mathrm{cat} : S \to C$ from concrete refusal scenarios S to categories:

$$C = \{\mathsf{Identity}, \mathsf{Capacity}, \mathsf{Topology}, \mathsf{Temporal}, \mathsf{Lifecycle}, \mathsf{Authorization}, \mathsf{Prerequisites}, \mathsf{Reserved}\}.$$

Definition 4.4 (Register Space and Operators)**.** Let $\mathbb{B}^8 = \{0,1\}^8$ be the space of 8-bit binary words (bytes). We define the following register operations on $x, y \in \mathbb{B}^8$:

- Bitwise AND: $x \wedge_8 y \in \mathbb{B}^8$.
- Bitwise OR: $x \vee_8 y \in \mathbb{B}^8$.
- Bitwise XOR: $x \oplus_8 y \in \mathbb{B}^8$.
- Bitwise NOT: $\neg_8 x \in \mathbb{B}^8$.
- Zero test: $\mathrm{z}(x) = 1$ if $x = 00000000_2$, and 0 otherwise.

- Non-zero test: $\mathrm{nz}(x) = 1$ if $x \neq 00000000_2$, and 0 otherwise.
- Population count: $\mathrm{pop}_8(x) \in \{0, 1, \ldots, 8\}$ counts the number of set bits in x.

The register operation alphabet is the set of these operators: $\mathcal{I}_8 = \{\wedge_8, \vee_8, \oplus_8, \neg_8, \mathrm{z}, \mathrm{nz}, \mathrm{pop}_8\}$.

Construction 4.2 (Machine Byte Algebra)**.** Let the standing byte be $b \in \mathbb{B}^8$. The admission gate is the operator $A_{\mathrm{gate}} : \mathbb{B}^8 \to \{0, 1\}$ defined as:

$$A_{\mathrm{gate}}(b) = \mathrm{z}(\neg_8 b)$$

For a mask $m \in \mathbb{B}^8$, the mask predicate $P_m(b) \in \{0, 1\}$ is defined as:

$$P_m(b) = \mathrm{z}((b \wedge_8 m) \oplus_8 m)$$

which evaluates to 1 if and only if all bits set in m are also set in b. The select operator $\mathrm{sel} : \mathbb{B}^8 \times \mathbb{B}^8 \to \mathbb{B}^8$ performs bitwise masking:

$$\mathrm{sel}(b, m) = b \wedge_8 m$$

Theorem 4.3 (Constant-Depth Eligibility)**.** *The gate, mask, and select operations compile to constant-depth expressions over register-level CPU operations.*

Proof. The gate operation $A_{\mathrm{gate}}(b)$ requires exactly two primitive register operations: NOT ($\neg_8$) and zero test (z). The depth is therefore bounded by 2. Similar analysis holds for the mask predicate $P_m(b) = \mathrm{z}((b \wedge_8 m) \oplus_8 m)$ with depth 3. Since $n = 8$ is fixed, all Boolean maps have constant-depth circuits. □

4.4 Operational Correspondence

The denial monoid is implemented as the `DenialPolarity` type in `crates/praxis-core/src/refusal.rs`. The refusal taxonomy is verified by a wildcard-free match on `RefusalScenario::category`. The standing byte is implemented in the `agent8` crate under `crates/agent8/src/byte.rs`.

4.5 Boundaries

No validation metadata outside the 8-bit standing vector is evaluated on the execution hot path.

Chapter Summary.

- **What was admitted:** The retraction α on $\mathcal{O}_\perp$, the monoid D, and the standing byte b.
- **What was refused:** Dynamic exceptions and variable-width validation pathways.
- **What this enables next:** The categories of lifecycle typestate transitions in Chapter 5.

Chapter 5

Chapter 5: Lifecycle Order: Typestate and Process

5.1 The Question

How do we ensure that every observation traverses the lifecycle in order, and that out-of-order operations fail at compile time?

5.2 Why the Question is Forced

This question is forced because runtime checks for step skipping add overhead and introduce failure states in execution. We must rely on the host language's type system to prove that illegal transitions are unconstructible.

5.3 Definitions and Constructions

Definition 5.1 (Lifecycle Category **Life**)**.** The lifecycle category **Life** is the free category on the linear quiver:

$$\mathsf{Raw} \xrightarrow{j} \mathsf{Validated} \xrightarrow{a} \mathsf{Admitted} \xrightarrow{r} \mathsf{Receipted},$$

where j is judgment, a is admission, and r is receipt.

Proposition 5.1 (Hom-Sets of **Life**)**.** *For any stages X, Y, the hom-set* $\mathbf{Life}(X, Y)$ *contains exactly one element if Y is reachable from X, and is empty otherwise.*

Proof. The quiver is a finite acyclic path with no parallel edges. In a free category, morphisms are paths. Since the path from X to Y is unique, there is at most one path. If Y precedes X, no directed path exists, so the hom-set is empty. □

Construction 5.1 (Factored Retraction)**.** The single-step retraction α is factored as:

$$\alpha = a \circ j,$$

where $j : \mathcal{O}_\perp \to \mathcal{O}_\perp \cup \{\perp\}$ transitions $\mathsf{Raw} \to \mathsf{Validated}$ by evaluating obligations, and $a : \mathcal{O}_\perp \cup \{\perp\} \to \mathcal{O}^* \cup \{\perp\}$ transitions $\mathsf{Validated} \to \mathsf{Admitted}$ by normalizing the payload. To

reconcile the category-theoretic and set-theoretic representations, we note that the admission map α is a retraction (an idempotent map $\alpha \circ \alpha = \alpha$) in the concrete set category **Set** where the objects are sets (namely $\mathcal{O}_\perp$) and the morphisms are functions. In contrast, **Life** is an abstract stage transition category represented as a free category over a linear quiver, where objects are abstract lifecycle stages and morphisms are allowed paths of execution.

Theorem 5.2 (Illegal Transitions are Uninhabited)**.** *For any two stages X, Y in the lifecycle category* **Life***, a transition path from X to Y is possible if and only if the hom-set* $\mathbf{Life}(X, Y)$ *is non-empty. If the hom-set* $\mathbf{Life}(X, Y) = \emptyset$*, then any transition from X to Y is uninhabited and mathematically impossible.*

Proof. By Proposition 5.1, the hom-set $\mathbf{Life}(X, Y)$ is empty if Y is not reachable from X along the directed edges of the quiver. Since morphisms in the free category **Life** represent all valid execution paths, the absence of a morphism (an empty hom-set) implies that no sequence of valid transitions can connect X to Y. Therefore, any transition attempting to bypass the quiver's order is uninhabited. □

Definition 5.2 (Q16.16 Format)**.** The Q16.16 format is a fixed-point number representation in which a 32-bit signed integer represents a real number by allocating the upper 16 bits to the integer part and the lower 16 bits to the fractional part. Specifically, an integer value $V \in \mathbb{Z}$ represents the real value $x = V \cdot 2^{-16}$. In this format, the unit interval $[0, 1]$ is represented by the integer range $[0, 65536]$.

Definition 5.3 (3-Node SEQ POWL Token Game)**.** The lifecycle process is a safe Petri net with state $M \in \{0, 1\}^4$ and transitions $\{j, a, r\}$. Let $M = (\texttt{TOK\_START}, \texttt{TOK\_JUDGED}, \texttt{TOK\_ADMITTED}, \texttt{TOK\_DONE})$. The conformance fitness $\varphi \in [0, 1]$ is computed in Q16.16 format based on token shortfall during replay.

Definition 5.4 (Pipeline Aggregate Denial Map)**.** Let Stage* be the free monoid on stages. The pipeline aggregate denial map $\Phi_o : \text{Stage}^* \to D$ evaluates the accumulated denial word over a sequence of stages:

$$\Phi_o(s_1 s_2 \cdots s_k) = \delta_{s_1}(o) \vee \delta_{s_2}(o) \vee \cdots \vee \delta_{s_k}(o)$$

where $\delta_s(o) \in D$ is the stage-denial map. The map is a monoid homomorphism satisfying $\Phi_o(uv) = \Phi_o(u) \vee \Phi_o(v)$ and $\Phi_o(\varepsilon) = \mathbf{0}$.

5.4 Operational Correspondence

The typestate is implemented via the sealed trait `Stage` on `LawObject` in `crates/praxis-core/src/lifecycle.r`
This corresponds directly to Theorem 5.2: stages are represented as Rust types, and transitions are methods consuming the object by value. Since the method for a transition $X \to Y$ is only defined when the object is in type X, attempting an out-of-order transition (such as Raw $\to$ Admitted without j) results in a compilation failure. The sealed trait design and module-private constructors ensure that the compiler's type checker statically guarantees that illegal transitions are uninhabited and cannot compile. Process replay is performed by the `PowlReplayVerifier` in `crates/praxis-core/src/replay_adapter.rs`.

5.5 Boundaries

Typestate enforcement is a static compiler boundary; no runtime checks are performed for path conformance in the hot path.

Chapter Summary.

- **What was admitted:** The category **Life** and the factored retraction $\alpha = a \circ j$.
- **What was refused:** Compiling stage transitions that bypass intermediate validation states.
- **What this enables next:** Formulating the deterministic manufacturing morphism in Chapter 6.

Chapter 6

Chapter 6: Manufacture: Deterministic Bounded Production

6.1 The Question

How do we define the compilation of admitted observations into artifacts such that the output is reproducible and bounded?

6.2 Why the Question is Forced

This question is forced because non-deterministic artifact generation prevents historical replay verification. Additionally, unbounded resource usage during manufacture would allow denial-of-service vectors.

6.3 Definitions and Constructions

Definition 6.1 (Manufacturing Morphism)**.** The manufacturing morphism $\mu : \mathcal{O}^* \to \mathcal{A}$, extended to the domain $\mathcal{O}^* \cup \{\bot\}$ by propagating the refusal constant such that $\mu(\bot) = \bot$, is a computable map satisfying:

(M1) **Determinism:** $\mu(x)$ depends only on x; equal admitted inputs yield bit-identical artifacts.

(M2) **Boundedness:** μ factors through a representation of size bounded by fixed structural constants, terminating with a priori bounded cost.

Proposition 6.1 (Single-Valuedness of Morphism)**.** *Under determinism (M1), μ is a mathematical function on $\mathcal{O}^*$, and the composite $\mu \circ \alpha$ is a function on $\mathcal{O}_\bot$ mapping to $\mathcal{A} \cup \{\bot\}$.*

Proof. By (M1), for any $x_1, x_2 \in \mathcal{O}^*$, $x_1 = x_2 \implies \mu(x_1) = \mu(x_2)$. Since α is single-valued, the composition of the function μ with α is a well-defined function. □

6.4 Operational Correspondence

The manufacturing morphism is implemented in the `bcinr-pddl` ontology compiler under `src/verbs/mfg.rs`, which compiles RDF graphs to PDDL domains deterministically.

6.5 Boundaries

The morphism μ cannot invoke dynamic or non-deterministic external network calls.

Chapter Summary.

- **What was admitted:** The deterministic, cost-bounded manufacturing morphism μ.
- **What was refused:** Non-deterministic or resource-unbounded execution paths during compilation.
- **What this enables next:** The generation of cryptographic causal chain receipts in Chapter 7.

Chapter 7

Chapter 7: Receipts: Commitment, Chain, and Replay

7.1 The Question

How can execution traces of arbitrary size be committed to a fixed-size representation that is verifiable in constant time?

7.2 Why the Question is Forced

This question is forced because we cannot transmit or parse large traces within the cognitive load limit κ. We must recursively hash execution frames to bind the entire history to a single hash value.

7.3 Definitions and Constructions

Definition 7.1 (Receipt)**.** A receipt $r \in \mathcal{R}$ is a tuple:

$$r = (\text{verdict}, h_+, \varphi, \text{reason}),$$

where verdict is a Boolean, h_+ is a 256-bit chain hash, φ is conformance fitness, and reason is a refusal scenario list.

Definition 7.2 (Concatenation and Frame Bracket Notation)**.** Let A, B be finite byte sequences. The concatenation operator $\|$ maps two byte sequences to their combined sequence $A \| B$ by appending B to the end of A. For metadata θ and a hash value $y \in \{0,1\}^{256}$, the frame bracket notation $\langle \theta, y \rangle$ represents a structured byte alignment that packs the fields into a canonical binary layout:

$$\langle \theta, y \rangle = \text{serialize}(\theta) \,\|\, y$$

where serialize(θ) is a fixed-length or prefix-free encoding of the metadata structure.

Construction 7.1 (Causal Chain Step)**.** Let $\mathsf{H} : \{0,1\}^* \to \{0,1\}^{256}$ be a collision-resistant hash function. For payload bytes b, metadata θ, and previous hash h_-, the frame is $\mathsf{fr} = \langle \theta, \mathsf{H}(b) \rangle$. The chained receipt hash is:

$$h_+ = \mathsf{H}(h_- \,\|\, \mathsf{fr}). \tag{7.1}$$

Lemma 7.1 (Chain Commitment)**.** *Under the collision resistance of* H*, the chain value* h_+ *is a binding commitment to the prefix history* (h_-, fr)*.*

Proof. Suppose a caller exhibits a distinct history (h'_-, fr') that yields the same hash h_+. This implies $\mathsf{H}(h_- \parallel \mathsf{fr}) = \mathsf{H}(h'_- \parallel \mathsf{fr}')$, which is a hash collision. Under the collision resistance assumption, the probability of this occurrence is negligible. By induction, h_+ binds all preceding frames in the chain. □

7.4 Operational Correspondence

The causal frame is implemented as `OcelCausalFrame` using BLAKE3 in `crates/praxis-core/src/law.rs`.

7.5 Boundaries

A receipt proves committed trace structure, not moral or semantic correctness.

Chapter Summary.

- **What was admitted:** The receipt tuple r and the recursive causal frame commitment.
- **What was refused:** Storing full execution trace histories directly on the verification hot path.
- **What this enables next:** Defining the system-wide invariants of the enforced equation in Chapter 8.

Chapter 8

Chapter 8: The Enforced Equation: BRCE

8.1 The Question

What invariants must a running system enforce to maintain the structural safety of the receipt calculus?

8.2 Why the Question is Forced

This question is forced because a system with unverified actuation paths would permit arbitrary unreceipted side effects, violating the security guarantees of the thesis.

8.3 Definitions and Constructions

Definition 8.1 (Bounded Receipted Chatman Equation)**.** A system satisfies the BRCE if for every actuated artifact $a \in \mathcal{A}$ it enforces four invariants:

(B1) **Admission Gate:** $a = \mu(x)$ for some $x = \alpha(o) \neq \bot$; no actuation occurs from raw or refused observations.

(B2) **Bounded Manufacture:** μ is deterministic and cost-bounded.

(B3) **Receipt Totality:** Every actuation appends exactly one chained receipt frame.

(B4) **Conformance:** The lifecycle replay has fitness $\varphi = 1.0$.

8.4 Operational Correspondence

The BRCE invariants are verified by the validation pipeline in `crates/praxis-core/src/verify.rs`.

8.5 Boundaries

Any execution trace that fails to maintain B1–B4 is halted before actuation.

Chapter Summary.

- **What was admitted:** The four invariants (B1–B4) of the BRCE framework.
- **What was refused:** Execution pathways that bypass the receipt and conformance checks.
- **What this enables next:** Proving the Conservation of Consequence in Chapter 9.

Chapter 9

Chapter 9: Conservation of Consequence

9.1 The Question

Can a consequence or action stand in the system without a verifiable, unique cause and receipt?

9.2 Why the Question is Forced

This question is forced because if consequence could be fabricated without a cause, the audit trail could be manipulated.

9.3 Definitions and Theorems

Definition 9.1 (Actuation-to-Terminal-Commitment Map)**.** The actuation-to-terminal-commitment map $\Psi : \mathcal{A} \to \{0,1\}^{256}$ associates each actuated artifact $a \in \mathcal{A}$ with its corresponding terminal receipt-chain hash $h_+ = \Psi(a)$ under the receipt totality invariant (B3).

Theorem 9.1 (Conservation of Consequence)**.** *Under* BRCE *invariants, the map $e \mapsto h_+(e)$ from actuation events (or trace steps) to receipt-chain positions (represented by receipt-chain hashes) is well-defined and injective up to hash collision, and the actuated artifact $a_e \in \mathcal{A}$ associated with each event e is caused by at least one admitted observation.*

Proof. We prove the two properties of the theorem separately.

(1) Causation. We prove by induction on the trace length N that for every step $t \in \{1, \ldots, N\}$, if an actuation event occurs yielding artifact a_t, then $a_t = \mu(x)$ for some admitted observation $x \in \mathcal{O}^*$. For the base case $N = 0$, the trace is empty; no actuation has occurred, so the claim holds vacuously. Assume that for all traces of length k, any actuated artifact a_t (for $t \leq k$) is the image under μ of some $x \in \mathcal{O}^*$. Consider a trace of length $k + 1$. By Receipt Totality (B3), if an actuation event e occurs at step $k + 1$, it must append exactly one receipt frame to the chain. The Admission Gate invariant (B1) is enforced at the gate before any actuation is finalized. By B1, the system transitions to a halted configuration if the input is refused, blocking actuation. Thus, actuation at step $k + 1$ is only executed if

the incoming observation o_{k+1} is successfully admitted, yielding $x_{k+1} = \alpha(o_{k+1}) \in \mathcal{O}^*$, and producing $a_{k+1} = \mu(x_{k+1})$. By induction, every actuated artifact in any trace of length N is caused by at least one admitted observation.

(2) Commitment Uniqueness. We show that the mapping from actuation events to receipt-chain positions is well-defined and injective. By Receipt Totality (B3), each actuation event e_t at step t maps to a unique receipt frame fr_t and a corresponding receipt-chain hash $h_+^{(t)}$, ensuring the map is well-defined. To prove injectivity, suppose two distinct actuation events e_{t_1} and e_{t_2} (with $t_1 < t_2$) map to the same receipt-chain hash, i.e., $h_+^{(t_1)} = h_+^{(t_2)}$. By Construction 7.1, the hash is computed as $h_+^{(t_2)} = \mathsf{H}(h_-^{(t_2)} \parallel \mathsf{fr}_{t_2})$. If $h_+^{(t_1)} = h_+^{(t_2)}$, we have $\mathsf{H}(h_-^{(t_1)} \parallel \mathsf{fr}_{t_1}) = \mathsf{H}(h_-^{(t_2)} \parallel \mathsf{fr}_{t_2})$. Under the collision-resistance assumption of H, this implies that $h_-^{(t_1)} \parallel \mathsf{fr}_{t_1} = h_-^{(t_2)} \parallel \mathsf{fr}_{t_2}$, which means the frames and prior histories must be identical. This contradicts the assumption that $t_1 < t_2$, as $h_-^{(t_2)}$ commits to $h_+^{(t_1)}$ recursively. Thus, the receipt-chain hash uniquely commits to the entire history, guaranteeing injectivity. □

Corollary 9.2 (The Antibody Clause)**.** *The overclaim set, defined as the set of actuations lacking a valid receipt, is empty by construction.*

9.4 Operational Correspondence

The conservation invariants are tested in the property tests under `crates/praxis-core/tests/prop_law.rs`. The program slogan is: "A grand system without receipts is grandiosity; a grand system that receipts its own limits is machinery."

9.5 Boundaries

The conservation guarantee is strictly conditional on the collision resistance of BLAKE3.

Chapter Summary.

- **What was admitted:** The Conservation of Consequence theorem separating causation from injectivity.
- **What was refused:** The assumption that μ must be injective to guarantee accountability.
- **What this enables next:** Tracing a worked contract-claim example in Chapter 10.

Chapter 10

Chapter 10: Worked Example: A Contract-Claim Law Object

10.1 The Question

How does a raw observation traverse the entire pipeline from submission to receipt?

10.2 Why the Question is Forced

This question is forced because we must verify the operational correctness of the type transitions and monoid composition.

10.3 Definitions and Constructions

Submission (Raw Stage). The caller submits a JSON observation payload:

$o = \{$ `"claim": "delivery-owed", "contract_id": "C-4471", "amount": 12000, "evidence": nul`

The system assigns three obligations: g_1 (contract active), g_2 (signed evidence present), and g_3 (amount within authority). The stage type is `LawObject<Claim, Raw, ContractLaw>`.

Judgment and Halt (Refusal Value). Upon evaluation, $g_1 = 1$ and $g_3 = 1$, but $g_2 = 0$ since evidence is null. The retraction maps $\alpha(o) = \bot$. The transition returns the halted object carrying the refusal metadata:

`Andon::Halted{ unmet: [EvidenceRequired], refusals: [MissingEvidence], at: 17188100 }`.

The denial monoid evaluates to $d(o) =$ `PRECONDITION_FAILED` $\neq \mathbf{0}$, halting the line.

Resubmission and Admission. The caller resubmits the payload with the signed evidence blob. All obligations pass: $\bigwedge_{i=1}^{3} g_i(o) = 1$. The object transitions to `Validated` via j, then to `Admitted` via a.

Manufacture and Receipt. The admitted payload $x \in \mathcal{O}^*$ compiles deterministically into the delivery record $a = \mu(x)$. The receipt consumes the object, hashing the frame and advancing the chain to h_+. The final receipt returned is:

$$r = (\text{verdict} = \text{true},\ h_+ = \texttt{0x7a2c...},\ \varphi = 1.0,\ \text{reason} = \varnothing).$$

10.4 Operational Correspondence

The worked example mirrors the integration tests in `crates/praxis-core/tests/prop_law.rs`. The agent's status transitions are projected into the `agent8` byte in `crates/agent8/src/byte.rs`.

10.5 Boundaries

Actuation is completely blocked while the object remains in the halted state.

Chapter Summary.

- **What was admitted:** The complete stage transition sequence for a contract claim.
- **What was refused:** Execution of PDDL domain compilation while the evidence obligation was failed.
- **What this enables next:** Mapping this foundational paper to the broader thesis program in Chapter 11.

Chapter 11

Chapter 11: Map of the Thesis Program

11.1 The Question

Where does this foundations paper sit within the broader Antigravity research series?

11.2 Why the Question is Forced

This question is forced because a single paper cannot address all cryptographic, geometric, and scaling dimensions of the receipt calculus.

11.3 Map of Pillars

Pillar I — Admission Algebra. This pillar deepens Chapter 4: the denial monoid, the refusal taxonomy as a total function, and the algebra of obligation composition.

Pillar II — Receipt Cryptography. This pillar deepens Chapter 7: the BLAKE3 causality frame chain, the Faithful Projection Theorem, and the affidavit verification pipeline.

Pillar III — Planning and Geometry. This pillar deepens Chapter 5 and 6: PDDL domain grounding, token-flow execution marking polytopes, and conformance as separating hyperplanes.

Pillar IV — The Projection Principle (Keystone). This pillar unifies the series, proving that verification cost is bounded at $O(\kappa)$ while system interior dimension grows.

Prior Work. The prior paper [1] demonstrated $\mathcal{A} = \mu(\mathcal{O})$ at scale using RDF/SHACL and KNHK executors.

11.4 Operational Correspondence

Downstream components map to the `ggen` project in `ggen.toml` and the `chatman-common` crate.

11.5 Boundaries

We do not analyze multi-agent game-theoretic trust dynamics; our scope is limited to single-system receipt boundaries.

Chapter Summary.

- **What was admitted:** The structural layout of the four downstream research pillars.
- **What was refused:** Attempting to prove cryptographic completeness within this foundations paper.
- **What this enables next:** Drawing final conclusions in Chapter 12.

Chapter 12

Chapter 12: Conclusion

12.1 The Question

What formal conclusions does the receipt calculus yield for autonomous agent safety?

12.2 Why the Question is Forced

This question is forced by the need to synthesize our proofs and state our operational limits.

12.3 Synthesis of Guarantees

We have formalised a receipt calculus that factors execution into admission, manufacture, and receipt. We have proven that semantic validation is impossible under Rice's Theorem, forcing syntactic retractions. We have proven that illegal lifecycle transitions are uninhabited type families. Finally, we have proven the Conservation of Consequence, guaranteeing that every actuated artifact is causally receipted. The standard was never comprehension; it is a lawful admission, a bounded manufacture, and a faithful receipt — each small enough to hold, each guaranteed by mechanism.

12.4 Operational Correspondence

The entire architecture is verified by the property tests and type checks in the `praxis` codebase.

12.5 Boundaries

Our formal guarantees are bounded by the collision-resistance strength of the BLAKE3 hash function.

Chapter Summary.

- **What was admitted:** The complete receipt calculus foundations.

- **What was refused:** The claim that any system can be verified by post-hoc explanations alone.
- **What this enables next:** The execution of downstream thesis program pillars.

Appendix A

Appendix A: Notation

This appendix defines the mathematical symbols and mappings. To resolve the inconsistencies identified in phase 2, we explicitly document the extended domain and factored mappings:

1. **Extended Observation Space:** $\mathcal{O}_\perp = \mathcal{O} \cup \mathcal{H} \cup \{\perp\}$ (incorporating halted configurations $\mathcal{H}$) is the domain of the admission map.
2. **Morphism Signature:** $\mu : \mathcal{O}^* \to \mathcal{A}$, extended to the domain $\mathcal{O}^* \cup \{\perp\} \to \mathcal{A} \cup \{\perp\}$ with $\mu(\perp) = \perp$.
3. **Factored Retraction:** $\alpha = a \circ j$ is the retraction map factored via intermediate stages in the set category **Set**.

Symbol	Meaning
$\mathcal{O}$	Raw observation space
$\mathcal{O}_\perp$	Extended observation space ($\mathcal{O} \cup \mathcal{H} \cup \{\perp\}$)
$\mathcal{O}^*$	Admitted syntactic subspace
α	Admission map / retraction ($\alpha : \mathcal{O}_\perp \to \mathcal{O}^* \cup \{\perp\}$)
$\perp$	Refusal constant (first-class value)
μ	Manufacturing morphism ($\mu : \mathcal{O}^* \to \mathcal{A}$, extended with $\mu(\perp) = \perp$)
$\mathcal{A}$	Artifact/action space
$\mathcal{R}$	Receipt space
H	Collision-resistant hash function (BLAKE3)
κ	Cognitive-load capacity limit ($\kappa \approx 4$)
φ	Conformance fitness ($\varphi \in [0, 1]$)
Life	Lifecycle category
BRCE	Bounded Receipted Chatman Equation invariants
Ψ	Actuation-to-terminal-commitment map ($\Psi : \mathcal{A} \to \{0,1\}^{256}$)
Φ_o	Pipeline aggregate denial map ($\Phi_o : \text{Stage}^* \to D$)

Table A.1: Table of Mathematical Symbols and Mappings

Abstract A (arXiv technical) and Abstract C (zero-background expository) are recorded in the program files under `docs/thesis/swarm_rewrite/ABSTRACTS.md`.

Appendix B

Appendix B: Code Correspondence

Every abstract mathematical object defined in this paper is anchored to a concrete implementation in the `praxis` codebase:

Mathematical Object	Code Artifact	Location
$\mathcal{O}$, raw observation	JSON payload check	`src/verbs/law.rs`
$\mathcal{O}_\perp$, extended space	`LawObject` input wrappers	`crates/praxis-core/src/law.rs`
α, factored retraction	`Judge::judge` and `Admit::admit`	`crates/praxis-core/src/law.rs`
$\perp$, refusal	`Andon::Halted` and `RefusalScenario`	`crates/praxis-core/src/law.rs`
D, denial monoid	`DenialPolarity` and `compose_denials`	`crates/praxis-core/src/refusal.rs`
Life, category	Sealed trait `Stage`	`crates/praxis-core/src/lifecycle.rs`
μ, morphism	PDDL compile verb	`src/verbs/mfg.rs`
h_+, receipt chain	`OcelCausalFrame` and BLAKE3 hash	`crates/praxis-core/src/law.rs`
φ, fitness	`PowlReplayVerifier`	`crates/praxis-core/src/replay_adapter`
BRCE, invariants	`verify` pipeline	`crates/praxis-core/src/verify.rs`
b, standing byte	`AgentByte` byte flags	`crates/agent8/src/byte.rs`

Table B.1: Code Correspondence Mapping

Appendix C

Appendix C: Claim Standing Table

This appendix lists the complete inventory of mathematical and logical claims extracted from the foundations manuscript, along with their standing status and verifying test suites:

Claim ID	Category	Status	LaTeX Location	Verifying Test / Code File
CLAIM-00_ABS_01	Definition	ADMIT	Abstract	crates/praxis-core/src/law.rs
CLAIM-00_ABS_02	Definition	ADMIT	Abstract	crates/praxis-core/src/law.rs
CLAIM-01_INT_03	Definition	ADMIT	Chapter 2	crates/praxis-core/src/law.rs
CLAIM-02_RIC_02	Theorem	ADMIT	Chapter 3	Theoretical (Theorem 3.1)
CLAIM-03_ADM_02	Theorem	ADMIT	Chapter 4	Verified (Theorem 4.1)
CLAIM-03_ADM_04	Impl Claim	ADMIT	Chapter 4	Andon::Halted representation
CLAIM-03_ADM_07	Theorem	ADMIT	Chapter 4	Verified (Proposition 4.2)
CLAIM-03_ADM_11	Impl Claim	ADMIT	Chapter 4	Wildcard-free category match
CLAIM-04_BYT_04	Theorem	ADMIT	Chapter 4	crates/agent8/benches/fleet_sweep.rs
CLAIM-05_LIF_02	Theorem	ADMIT	Chapter 5	Verified (Proposition 5.1)
CLAIM-05_LIF_05	Theorem	ADMIT	Chapter 5	Verified (Theorem 5.2)
CLAIM-06_MFG_03	Theorem	ADMIT	Chapter 6	Verified (Proposition 6.1)
CLAIM-06_MFG_08	Theorem	ADMIT	Chapter 7	Verified (Lemma 7.1)
CLAIM-06_MFG_11	Theorem	ADMIT	Chapter 9	Verified (Theorem 9.1)
CLAIM-06_MFG_13	Theorem	ADMIT	Chapter 9	Verified (Corollary 9.2)

Table C.1: Claim Standing and Verification Table

Bibliography

[1] S. Chatman, *The Chatman Equation and the Industrial Revolution of Knowledge: $A = \mu(O)$, Knowledge Hooks, and Production-Verified Enterprise Execution*, v1.2.0, November 2025.

[2] H. G. Rice, *Classes of recursively enumerable sets and their decision problems*, Transactions of the American Mathematical Society **74** (1953), 358–366.

[3] R. E. Strom and S. Yemini, *Typestate: A programming language concept for enhancing software reliability*, IEEE Transactions on Software Engineering **SE-12**(1) (1986), 157–171.

[4] *Workflow patterns*, Distributed and Parallel Databases **14**(1) (2003), 5–51.

[5] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn, *BLAKE3: One function, fast everywhere*, 2020.

[6] A. F_Ghahfarokhi, G. Park, A. Berti, and W. M. P. van der Aalst, *OCEL: A standard for object-centric event logs*, in New Trends in Database and Information Systems, Springer, 2021.

[7] G. A. Miller, *The magical number seven, plus or minus two: some limits on our capacity for processing information*, Psychological Review **63**(2) (1956), 81–97.

[8] N. Cowan, *The magical number 4 in short-term memory: A reconsideration of mental storage capacity*, Behavioral and Brain Sciences **24**(1) (2001), 87–114.